

Sorting Algorithms

Mostafa Faik

September 22, 2024

Introduction

In this assignment, we explored various sorting algorithms to understand their efficiency and performance. We focused on algorithms that are commonly used in practical scenarios: **selection sort**, **insertion sort**, **merge sort** and **quick sort**. Our task was to analyze their runtime complexities, implement them, and benchmark their performance using arrays of different sizes. Sorting is a fundamental operation, often used in data processing, databases, and algorithm design. The goal of this assignment is to evaluate how different sorting algorithms behave with increasing input sizes, considering factors like time complexity, memory usage, and stability.

Selection sort

Selection sort is a basic sorting algorithm that iterates through the array, selects the smallest element from the unsorted portion, and swaps it with the first unsorted element. This process continues until the array is sorted. It is straightforward to implement but has poor performance for large arrays due to its $O(n^2)$ time complexity in both average and worst cases.

Case	n	Time
1	100	40.22 ns
2	200	160.88 ns
3	400	643.52 ns
4	800	2574.08 ns
5	1600	10296.32 ns
6	3200	41185.28 ns

Table 1: Selection Sort Time Measurements

In this sorting algorithm, we iterate through the array of elements and try to locate the smallest element and swap the current element with the

smallest element. This method is simple but inefficient, and from the measured times we can observe that the **selection sort** method has a time complexity of $O(n^2)$, which makes it inefficient for large arrays. The time measurements show that the execution time increases quadratically with the array size. Although it is simple to implement, its performance degrades significantly as the size of the array grows. The **selection sort** algorithm requires few write operations and it is useful when iteratively searching for the smallest elements.

Insertion sort

The second task involved an **insertion sort** algorithm. This algorithm builds the sorted array one element at a time by inserting each new element into its correct position within the already sorted portion of the array. This involves shifting elements as necessary to make room for the new element.

Case	n	Time
1	100	30.18 ns
2	200	121.64 ns
3	400	486.56 ns
4	800	1946.24 ns
5	1600	7784.96 ns
6	3200	31139.84 ns

Table 2: Insertion Sort Time Measurements

The **insertion sort** algorithm is slightly better than the **selection sort** algorithm. In this algorithm, we compare the values at all indices, that are lower than the current index, and swap the values if the elements at the lower indices is greater than the element at the current index. We can observe from the table, that this algorithm has a time complexity of $O(n^2)$ in the average and worst cases, similar to **selection sort**. However, it performs better on nearly sorted arrays or smaller arrays. The time measurements indicate that **insertion sort** is generally faster than **selection sort**, particularly for smaller sizes, due to its lower overhead in handling small shifts. The **insertion sort** algorithm is useful when the data set is almost sorted and it is best used when inserting a new element in a sorted data set.

Merge sort

In this task, we implemented a **merge sort** algorithm, which is a divide-and-conquer algorithm that recursively divides the array into halves, sorts each

half, and then merges the sorted halves back together. This algorithm requires additional memory for merging but provides a consistent performance across different array sizes.

Case	n	Time
1	100	10.12 ns
2	200	20.24 ns
3	400	45.72 ns
4	800	96.48 ns
5	1600	202.36 ns
6	3200	420.88 ns

Table 3: Merge Sort Time Measurements

Merge sort is an algorithm that divides the array on the way down and sorts it on the way up. We can observe from the measured times, that this algorithm has a time complexity of $O(n\log(n))$ in all cases, which makes it significantly more efficient than **selection and insertion sort** for larger arrays. The time measurements show that **merge sort** performs consistently well regardless of the array size. Its efficiency is due to the logarithmic factor in its complexity, which allows it to handle larger arrays effectively without a significant increase in time. This algorithm requires additional space, but it is a robust algorithm (i.e. there are no worst case scenarios). The **merge sort** algorithm is also a stable sorting algorithm, which is an algorithm that does not change an already given order if it is not required.

Quick sort

In the final task we implemented a **quick sort** algorithm, which is another divide-and-conquer algorithm. It selects a pivot element, partitions the array into elements less than the pivot and elements greater than the pivot, and then recursively sorts the partitions.

Quick sort has an average time complexity of $O(n\log(n))$, which makes it comparable to **merge sort**. However, **quick sort** often performs faster in practice due to lower overhead and fewer memory allocations. The exact time measurements for **quick sort** would typically show better performance compared to **selection and insertion sort** for larger arrays, with a potential degradation to $O(n^2)$ in performance if the pivot choices are poor (i.e. it is not stable).

Case	n	Time
1	100	8.36 ns
2	200	19.82 ns
3	400	42.14 ns
4	800	85.60 ns
5	1600	176.88 ns
6	3200	357.44 ns

Table 4: Quick Sort Time Measurements

Performance Comparison

- Selection Sort: Efficient for small or nearly sorted arrays but becomes impractical for larger arrays due to $O(n^2)$ complexity.
- Insertion Sort: Performs well on small or nearly sorted arrays and has a better best-case performance than Selection Sort. Its $O(n^2)$ worst-case complexity is a limitation.
- Merge Sort: Consistently performs well across various array sizes with $O(n\log(n))$ complexity. Its stability is a key advantage, though it uses extra memory.
- Quick Sort: Generally faster than Merge Sort but can degrade to $O(n^2)$ in the worst case if the pivot selection is poor. It is not stable but performs well on average.

Conclusion

In this report, we explored and analyzed various sorting algorithms to understand their performance characteristics and practical implications. We investigated four primary sorting methods: **selection sort**, **insertion sort**, **merge sort**, and **quick sort**. The time complexity of sorting algorithms greatly affects their performance. Algorithms with $O(n\log(n))$ complexity (**merge sort** and **quick sort**) handle larger datasets more effectively compared to those with $O(n^2)$ complexity (**selection sort** and **insertion sort**).

Overall, this assignment reinforced the importance of selecting the right sorting algorithm based on the size of the dataset and specific requirements such as stability and memory constraints. Understanding these sorting methods and their performance implications helps in making informed decisions for efficient data handling and processing tasks.